NUTRIMIND AI

Meal Plan AI Agent – Nutrition and Recipe Tools

Group members: Y.D.Sasmitha Methhasindu (cclarke667)

P.Dulanjana Thathsarani (cclarke527)

Course: Higher Diploma in AI and Software Engineering

Module: Large Language Models

Lecturer: Yashod Perera

Institute: C Clarke International Institute of Digital Science

Date of Submission: 25th October 2026

Abstract

This Meal Plan AI Agent is a Python Application that allows users to plan their meals based on their nutritional values, diet preferences, allergies, ingredient cost, and storage requirements. The program combines a range of tools to suggest recipes, calculate nutritional values, estimate ingredient cost, filter by dietary requirements, provide storage suggestions, and store the meal plan.

This application has a modular structure; each Python File handles a specific function within the application. The agent controls the tools; there is a JSON file to store ingredient prices and an SQLite database to store the meal plans. Now this application does not use any Machine Learning Models; it uses rule based reasoning and static functions. This makes the application relatively easy to develop, test, and maintain.

This report will explore the scenario, architecture, agents, and tools used, the agent loop, prompts, testing, challenges, and limitations. It also outlines the improvements that can be made in the future, and my contribution. This report looks into why accurately calculating nutritional values, filtering allergies, and error handling are important.

Table of contents

1. Introduction: Problem and Scenario.....04
2. System Architecture.....05
   2.1 Main Components.....05
   2.2 Data Flow.....06
3. Agents and Tools.....06
   3.1 Recipe Generation Tool.....06
   3.2 Nutrition Calculation Tool.....07
   3.3 Allergy and Dietary Restriction.....07
   3.4 Budget Calculation Tool.....08
   3.5 Food Storage Tool.....08
   3.6 Database Tool.....08
4. Agent Loop and Workflow.....09
5. Prompt Design and Input Handling.....09
6. Testing and Evaluation.....10
   6.1 Unit Testing.....10
   6.2 Integration Testing.....10
   6.3 Suggested Test Cases.....11
7. Development Changes.....11
8. Limitations.....12
9. Future Improvements.....12
10. Individual Contributions.....13
11. Conclusion.....13

1.Introduction: Problem and Scenario

Meal planning is an important part of maintaining a balanced diet. However, selecting suitable meals can be difficult when users need to consider nutrition, personal preferences, allergies, dietary restrictions, and budget limitations simultaneously.

For example, a user may want a vegetarian meal containing sufficient protein, with no dairy products, that costs less than a specified amount. Manually checking recipes, calculating nutritional values, comparing ingredient prices, and identifying unsuitable ingredients can take considerable time.

The Meal Plan AI Agent aims to simplify this process by combining these tasks into one application. Users provide their requirements, and the system processes the information to produce a meal suggestion with nutritional estimates, ingredient costs, dietary screening results, and storage guidance.

The main objectives are:

1. Generate recipe suggestions based on user preferences.
2. Calculate calories, protein, carbohydrates, and fat.
3. Check recipes against allergies and dietary restrictions.
4. Estimate ingredient costs using stored prices.
5. Provide basic food storage recommendations.
6. Save and retrieve meal plans using a database.
7. Organize the application into reusable and maintainable modules.

The system is intended as a meal-planning support tool, not as a replacement for professional nutritional advice or medical assessment.

2.System Architecture

The application uses a modular architecture. Each module has a clearly defined responsibility, allowing developers to modify or test one component without rewriting the entire application.

The proposed project structure is:

```
meal_plan_ai_agent/
│
├── agent.py
├── database.py
├── nutrition_tool.py
├── recipe_tool.py
├── budget_tool.py
├── storage_tool.py
│
└── prices.json
```

The exact filenames should match the final project implementation.

2.1 Main Components

agent.py: This is the central controller. It receives user requirements, calls the appropriate tools, checks their outputs, and prepares the final meal-plan response.

recipe_tool.py: This module generates or selects recipes using available ingredients and predefined rules.

nutrition_tool.py: This module calculates estimated calories, protein, carbohydrates, and fat from ingredient quantities and nutritional data.

budget_tool.py: This module calculates estimated ingredient costs using prices stored in the JSON file.

storage_tool.py: This module provides basic food storage guidance for recognized ingredients.

database.py: This module manages the SQLite database, including table creation, saving meal plans, and retrieving previously stored plans.

prices. Json: This file contains ingredient price information. Keeping prices outside the Python code makes it easier to update without modifying the main program.

2.2 Data Flow

The system begins by receiving the user's requirements. The agent sends the request to the recipe tool, which produces a candidate recipe. The dietary checker then examines the ingredients for conflicts with the user's stated requirements.

If the recipe passes the required checks, the nutrition and budget tools calculate their respective results. The storage tool supplies relevant recommendations, and the database module saves the plan when required.

Finally, the agent combines the outputs into a readable response. If an ingredient restriction is violated or important information is missing, the agent should report the problem rather than incorrectly presenting the recipe as suitable.

3. Agents and Tools

An agent coordinates actions to achieve a goal. In this project, the main agent manages a sequence of specialized tools rather than performing every calculation inside one large function.

The current implementation is best described as a rule-based agent because its behavior depends on predefined conditions and programmed logic. It does not automatically learn from experience unless learning functionality is added separately.

3.1 Recipe Generation Tool

The recipe tool generates or selects meals based on available ingredients and user preferences. For example, it can filter out meat when a vegetarian meal is requested.

A vegan meal must exclude animal-derived ingredients such as meat, fish, eggs, and dairy products. The tool uses predefined rules to identify ingredients that are compatible with the requested diet.

The quality of the suggestions depends on the available recipe information and the completeness of the rules. The current approach is suitable for a small prototype, but a larger application would benefit from a more extensive recipe database.

3.2 Nutrition Calculation Tool

The nutrition tool calculates the nutritional contribution of each ingredient and adds the results together.

The basic formula is:

Nutrient contribution = (Ingredient quantity in grams ÷ 100) × Nutrient value per 100 grams

For example, if 200 grams of an ingredient contains 130 calories per 100 grams, the estimated contribution is:

(200 ÷ 100) × 130 = 260 calories.

The tool repeats this calculation for each ingredient and calculates the total calories, protein, carbohydrates, and fat for the recipe.

The accuracy of the output depends on the supplied nutrition data. Raw and cooked ingredients must be distinguished because their nutritional values can differ due to changes in water content and preparation.

3.3 Allergy and Dietary Restriction Tool

The allergy and dietary restriction checker compares recipe ingredients with the user's stated requirements.

It can screen for common allergens such as milk, eggs, peanuts, tree nuts, fish, shellfish, soy, and wheat. It can also check requirements such as vegetarian, vegan, dairy-free, gluten-free, and egg-free diets.

For example, a recipe containing milk should be flagged for a dairy-free user, while a chicken recipe should be rejected for a vegetarian.

This check must happen before the agent presents the recipe as suitable. However, simple ingredient matching cannot guarantee allergy safety. Hidden ingredients, compound products, inaccurate ingredient data, and cross-contact may not be detected. Users with serious allergies must verify product labels and preparation conditions.

3.4 Budget Calculation Tool

The budget tool estimates the cost of the ingredients needed for a recipe. Ingredient prices are stored in prices. Json, allowing the application to calculate costs without connecting to a live shopping service.

If the price is recorded per kilogram, the formula is:

Ingredient cost = (Quantity in grams ÷ 1,000) × Price per kilogram

For example, if an ingredient costs LKR 400 per kilogram and the recipe requires 250 grams, the estimated cost is:

(250 ÷ 1,000) × 400 = LKR 100.

The tool calculates each ingredient's cost and adds the results to estimate the total recipe cost. This amount can then be compared with the user's budget.

The result is an estimate rather than a guaranteed purchase price because market prices, package sizes, waste, and other costs may vary.

3.5 Food Storage Tool

The storage tool provides basic guidance for storing ingredients correctly. For example, perishable foods such as meat, fish, and milk generally require refrigeration, while dry ingredients may be stored in clean, dry, sealed containers.

The tool uses predefined information associated with recognized ingredients. The recommendations should remain general unless reliable; ingredient-specific storage information is available.

3.6 Database Tool

The database module uses SQLite to save meal plans. It can store the recipe name, ingredients, nutritional totals, estimated cost, and date of creation.

Functions such as create_table(), save_plan(), and get_plans() can separate database operations from the agent's decision-making logic.

SQLite is appropriate for this project because it is lightweight and does not require a separate database server. Saving plans also allows the user to retrieve previous results instead of generating them again.

4. Agent Loop and Workflow

The agent loop describes how the application moves from the user's request to the completed result. In this project, a fixed workflow provides predictable behaviour and makes testing easier.

The process follows these steps:

1. **Receive input:** Collect the user's dietary preferences, allergies, budget, and meal requirements.
2. **Validate input:** Check that required fields and quantities are meaningful.
3. **Generate a recipe:** Select or construct a candidate meal using the available ingredients.
4. **Check restrictions:** Compare the ingredients with allergy and dietary requirements.
5. **Calculate nutrition:** Estimate the total calories, protein, carbohydrates, and fat.
6. **Calculate cost:** Estimate ingredient costs and compare the total with the budget.
7. **Retrieve storage guidance:** Add relevant recommendations for the ingredients.
8. **Save the plan:** Store the result in SQLite when saving is required.
9. **Return the result:** Present the recipe and its estimates, along with warnings or missing information.

The workflow must have clear stopping conditions. If no suitable recipe is available, the agent should explain that it could not find a valid option. It should not repeatedly search without a limit.

Allergy conflicts must be treated as strict restrictions rather than preferences that can be ignored because a recipe is cheap or nutritious.

5. Prompt Design and Input Handling

Prompt design is important when a project uses a language model to understand user requests or generate explanations. If the implementation is entirely rule-based, these principles can still guide the input and output structure, but the project should not claim to use a language model unless one is integrated.

A suitable instruction for a language-model-based version would be:

“You are a meal-planning assistant. Use the available recipe, nutrition, dietary-checking, budget, storage, and database tools as appropriate. Treat stated allergies as strict exclusions. Use supplied data for calculations, show estimated values with units, and identify missing

information. Do not claim that a recipe is medically safe or free from cross-contact. If no suitable recipe is available, explain the reason rather than inventing a result."

The input should be normalized before processing. For example, terms such as "no dairy" and "dairy-free" should map to the same restriction.

The output should contain the recipe name, ingredient quantities, nutritional estimates, estimated cost, dietary-checking results, storage advice, and relevant warnings.

Missing prices or nutrition values should not be silently replaced with zero. The system should identify unavailable information so users can understand the reliability of the result.

6. Testing and Evaluation

Testing is necessary to confirm that each module behaves correctly and that the agent coordinates them as intended.

6.1 Unit Testing

Individual functions should be tested separately.

- **Nutrition tool:** Compare calculated values with manually calculated results.
- **Budget tool:** Verify quantity conversions and ingredient cost totals.
- **Recipe tool:** Check whether it returns suitable ingredients for the selected preferences.
- **Restriction checker:** Verify that restricted ingredients are identified.
- **Database tool:** Confirm that meal plans can be saved and retrieved.
- **Storage tool:** Confirm that recognized ingredients return appropriate guidance.

6.2 Integration Testing

Integration testing checks whether the modules work together. For example, a generated recipe should pass through the restriction checker before nutrition and budget results are presented as a recommended plan.

A useful test scenario is a user requesting a vegetarian, dairy-free meal under a specified budget. The system should generate a candidate recipe, screen its ingredients, calculate the nutrition and cost, and provide storage recommendations.

6.3 Suggested Test Cases

Test case	Expected result
Recipe contains milk; user requests dairy-free	Recipe is rejected or flagged
Chicken recipe; user requests vegetarian	Recipe is rejected
Recipe cost is below the budget	Budget check passes
Recipe cost exceeds the budget	Budget warning is displayed
Ingredient price is missing	Cost is marked incomplete or unknown
Quantity is negative	Input is rejected
Recipe has known nutrition data	Nutrient totals are calculated
Meal plan is saved to SQLite	Saved record can be retrieved

Actual test results should be recorded after running the final program. These cases are a proposed testing plan and should not be reported as passed unless they have been executed successfully.

7. Development Challenges

One challenge is maintaining consistent ingredient names across the recipe tool, nutrition data, restriction lists, and price files. Different terms for the same ingredient may cause incorrect matching. A consistent naming system and synonym mapping can reduce this problem.

Another challenge is maintaining accurate nutritional calculations. Raw and cooked foods may have different nutritional values, so the system must use the correct data for the quantity being measured.

Price calculations also require consistent units. If a price is stored per kilogram, but the recipe quantity is expressed in grams, the program must convert the units correctly.

Handling missing information is another important challenge. Missing prices should not be treated as free ingredients, and unclear allergy information should not automatically be treated as safe. The system should also handle invalid JSON, unexpected user inputs, and database errors.

These challenges demonstrate why validation, clear function interfaces, and separate modules are important in software development.

8. Limitations

The system depends on the quality and completeness of its stored data. Ingredient prices may become outdated, and nutritional values may not represent every brand or preparation method.

Recipe generation is limited by the rules and ingredients available in the program. A simple restriction checker may miss hidden allergens, ingredient synonyms, and cross-contact during preparation.

The application does not automatically verify medical suitability, obtain live market prices, or guarantee food safety. Its storage advice may also require additional details for particular ingredients.

The current implementation should therefore be considered a prototype and decision-support tool rather than a complete autonomous AI system or professional nutrition service.

9. Future Improvements

Several improvements could make the system more useful.

First, a graphical user interface could make it easier to enter preferences, choose meal types, and view results. Weekly meal planning and shopping-list generation could also be introduced.

Second, a more complete ingredient database could include nutritional values, common ingredient names, allergens, portion sizes, and raw or cooked states.

Third, the budget tool could support dated prices, different shops, and package sizes. This would improve cost estimates and make them more useful for real shopping.

Fourth, allergy screening could be strengthened through structured ingredient categories, synonym mapping, and clearer warnings about label verification and cross-contact.

Fifth, automated testing and error logging could improve reliability. The database could also support user profiles and saved preferences, with suitable privacy protections.

Finally, a language model could be added to interpret natural-language requests and explain recommendations. However, numerical calculations and strict dietary checks should remain

controlled by reliable program functions. A language model should never override a failed allergy check.

10. Individual Contributions

The contribution section should accurately reflect the work completed by each team member. The project development work discussed includes the agent controller, recipe and nutrition functions, dietary restriction checking, budget calculation, storage guidance, the database module, and the JSON price file.

The exact allocation of tasks among individual team members must be confirmed before submission.

For an individual project, the following statement can be adapted:

“I contributed to the design and development of the Meal Plan AI Agent using Python. My work involved implementing and connecting the required modules for recipe handling, nutrition calculations, budget estimation, dietary restriction checking, and data storage. I also considered input validation, missing-data handling, testing requirements, and possible future improvements.”

Only retain the tasks that you personally complete. For a group project, list each member's assigned module, completed tasks, and supporting evidence, such as code commits, test records, or documentation.

11. Conclusion

The Meal Plan AI Agent demonstrates how Python modules can be combined to support recipe planning, nutrition estimation, dietary screening, cost calculation, storage guidance, and database persistence.

The modular architecture separates the main responsibilities, while JSON and SQLite provide simple methods for managing ingredient prices and saved meal plans. The rule-based agent coordinates the tools through a defined workflow, making the project relatively easy to understand, maintain, and test.

Although the prototype has limitations relating to data accuracy, recipe coverage, and allergy screening, it provides a useful foundation for further development. Improving the ingredient database, strengthening validation, carrying out systematic tests, and enhancing the user interface could make the application more reliable and practical.